

Tokens in AI: How Machines Read Text

1. Introduction

When we talk to AI models like ChatGPT, it feels like we're just sending normal text: sentences, paragraphs, maybe a long prompt.

But under the hood, the model doesn't actually "see" full words and sentences the way we do.

It sees everything as **tokens**.

Understanding tokens is important for three reasons:

- It explains **how** models read and process text.
- It explains **why there are limits** on how much you can send in one go.
- It explains **why you're charged** a certain amount when using APIs.

In this session, we'll break down what tokens are, how tokenization works, and why they matter for things like cost, performance, and prompt design.

2. Why Tokens Matter

Think of an AI model as a very fast pattern-matching machine.

It doesn't understand "meaning" the way humans do. Instead, it learns patterns over sequences of tiny pieces of text. Those tiny pieces are tokens.

Why this matters:

- The model never sees your prompt as one big string. It sees it as a **sequence of tokens**.
- Every step in its reasoning is: *"Given the previous tokens, what token comes next?"*
- The **length limit** of the model is defined in tokens, not words or characters.
- The **cost** of using the model (in APIs) is also based on tokens.

So if you care about context limits, latency, or pricing, you automatically care about tokens—even if you never see them directly.

3. What Is a Token?

A **token** is just a chunk of text.

Depending on the language and the tokenizer, a token can be:

- A whole word: cat
- A part of a word: inter, national, ization
- Punctuation: , or ?
- Special symbols or spaces

For example, take the sentence:

“I’m learning about tokens in AI.”

A tokenizer might split this into something like:

- I
- ’m
- learning
- about
- tokens
- in
- AI
- .

Notice how “**I’m**” becomes two tokens: I and ’m.

This is done because subword pieces are easier for the model to reuse across many different words and languages.

The important idea:

The model doesn’t know “words” in the way a dictionary does. It only knows **patterns in token sequences**.

4. How Tokenization Works

Tokenization is the process of converting raw text into tokens.

When you send text to a model:

1. Your text is passed through a **tokenizer**.
2. The tokenizer breaks the text into tokens according to its rules.
3. Each token is converted into a unique **number ID**.
4. The model works only with those numbers.

Different languages and scripts tokenize differently:

- In English, you might get around 3–4 characters per token on average.
- In languages with more complex scripts, the ratio can be different.
- Emojis, punctuation, and symbols are all treated as tokens.

The tokenizer is fixed when the model is trained.

That means for a given model, the same text will always become the same sequence of tokens.

5. Types of Tokenization (High Level)

There are different strategies for tokenization. At a high level, you'll hear about:

1. **Word-level tokenization**
 - a. Each word is a token.
 - b. Simple, but breaks with misspellings, rare words, and new slang.
2. **Character-level tokenization**
 - a. Each character is a token.
 - b. Very flexible, but sequences get extremely long and less efficient.
3. **Subword-level tokenization** (most common in modern LLMs)
 - a. Words are broken into frequently occurring **subword units**.
 - b. Examples: Byte Pair Encoding (BPE), WordPiece.

- c. This handles common words efficiently, but can also represent rare or new words by combining subwords.

Example: the word “**tokenization**” might be split as:

- token
- ization

The model sees these subpieces across many words: organization, realization, globalization, etc.

That helps it generalize better and compress vocabulary.

6. Tokens in Large Language Models

Large language models operate on **sequences of tokens**.

Here’s the flow when you send a prompt:

1. Your text → tokenizer → list of token IDs.
2. The model takes these token IDs and processes them through many layers.
3. At each step, it predicts the **next token** in the sequence.
4. Those predicted tokens are then decoded back into text.

Two key concepts here:

- **Context window**

This is the maximum number of tokens the model can handle at once.

It includes both:

- Your input (the prompt)
- The model's output (the completion)

If your prompt is too long, the model may **truncate** earlier parts—meaning it literally cannot see the beginning anymore.

- **Autoregressive prediction**

The model generates text one token at a time, based only on the tokens it has seen so far.

So, when people say “this model supports X tokens of context,” they’re talking about **token count**, not the number of words or characters.

7. Tokens, Cost, and Performance

Most API providers for LLMs charge based on **tokens**, not requests.

You usually pay per **1,000 tokens**, and that count includes:

- Tokens in your prompt
- Tokens in the model's response

This leads to a few practical implications:

- Long prompts + long responses = more tokens = higher cost.
- Repeating the same context over and over in the prompt can become expensive.
- Very long prompts also run into the **context limit**, which can cause older parts of the conversation to be dropped.

From a performance angle:

- More tokens = more compute = higher latency.
- Optimizing your prompts for clarity and brevity gives you better efficiency without losing quality.

So, good prompt design is partly about **being smart with tokens**: say enough to be clear, but not so much that you waste context and money.

8. Tokens in Training vs Inference

Tokens show up in two phases of a model's life:

1. Training phase

- a. The model is trained on massive datasets, which are all converted into tokens.
- b. It learns: "Given tokens A, B, C... what is the most likely next token?"
- c. This happens billions or trillions of times across many token sequences.

2. Inference phase (when you use the model)

- a. The same tokenizer is used.
- b. Your text is converted into tokens, and the model predicts the next tokens in real time.
- c. The behavior you see is the result of the patterns it learned from all those training tokens.

The key point:

Training = learning patterns from huge streams of tokens.

Inference = applying those patterns to your specific tokens.

9. Tokens and Embeddings (Quick View)

Another place where tokens matter is **embeddings**.

- An **embedding** is a vector (a list of numbers) that represents the meaning of a token, a word, a sentence, or a document.
- To get embeddings, text is again tokenized first.
- Those tokens are then transformed into embeddings by a model.

You can think of it like this:

Token → Embedding → Model uses these numbers to compare meanings.

These embeddings are used for:

- Semantic search
- Clustering similar texts
- Recommendation systems
- Retrieval-augmented generation (RAG), where you fetch relevant chunks before answering

All of that still rests on the foundation of consistent tokenization.

10. Practical Tips: Working Smart with Tokens

If you're designing prompts, chat flows, or AI features, here are some practical tips:

1. Be concise but specific

- a. Remove unnecessary fluff, but keep the details that matter.
- b. “Summarize this for a sales manager who needs action points” is better than “Summarize this.”

2. Avoid repeating large blocks of text

- a. If the model already “knows” a chunk in the context, refer back to it instead of pasting it again, when possible.

3. Structure your input

- a. Use headings, bullet points, and clear segments.
- b. This doesn’t change token count a lot, but it makes patterns easier for the model to follow.

4. Be aware of context limits

- a. Very long chat histories will eventually be truncated.
- b. For important information, consider re-stating key points briefly rather than relying on very old context.

5. Monitor cost and latency

- a. If usage goes up, check how many tokens your prompts and responses are consuming.
- b. Small changes to prompt design can have a big impact on token usage at scale.

11. Conclusion

To wrap up:

- AI models don’t see text as words and sentences. They see **tokens**.
- Tokenization is the process of breaking text into those tokens.
- Everything—from context limits, to cost, to performance—is measured in **tokens**, not words.
- During training and inference, the model learns and operates purely on token sequences.
- If you understand tokens, you can design better prompts, control cost, and build more efficient AI features.

If you remember only one thing, let it be this:

Every interaction with a large language model is really a conversation in tokens—your job is to use those tokens wisely.